

7. String handling

- 1.String, String constructor
- 2.String operations:
 - String extractions
 - String Comparison
 - Searching strings
 - Modifying a string
 - To String() and valueOf() methods
- 3.String Buffer, String Buffer Constructor
 - String Buffer operations & methods

Java String

What is String in java?

- Generally, string is a sequence of characters. But in java, string is an object that represents a sequence of characters.
- String class is used to create string object.
- Java String provides a lot of functionalities that can be performed on a string such as compare, concat, equals, split, length, replace, compareTo, intern, substring etc.
- In java, string is basically an object that represents sequence of char values.

Example:

```
char[] ch={'j','a','v','a'};  
String s=new String(ch);  
  
is same as  
  
String s="java";
```

- The `java.lang.String` class implements *Serializable*, *Comparable* and *CharSequence* interfaces.
- The java String is immutable i.e. it cannot be changed but a new instance is created.
- For mutable class, you can use `StringBuffer` and `StringBuilder` class.

How to create String object?

There are two ways to create String object:

1. By string literal
2. By new keyword

1) String Literal

Java String literal is created by using double quotes.

```
String s="welcome";
```

Each time you create a string literal, the JVM checks the string constant pool first. If the string already exists in the pool, a reference to the pooled instance is returned. If string doesn't exist in the pool, a new string instance is created and placed in the pool.

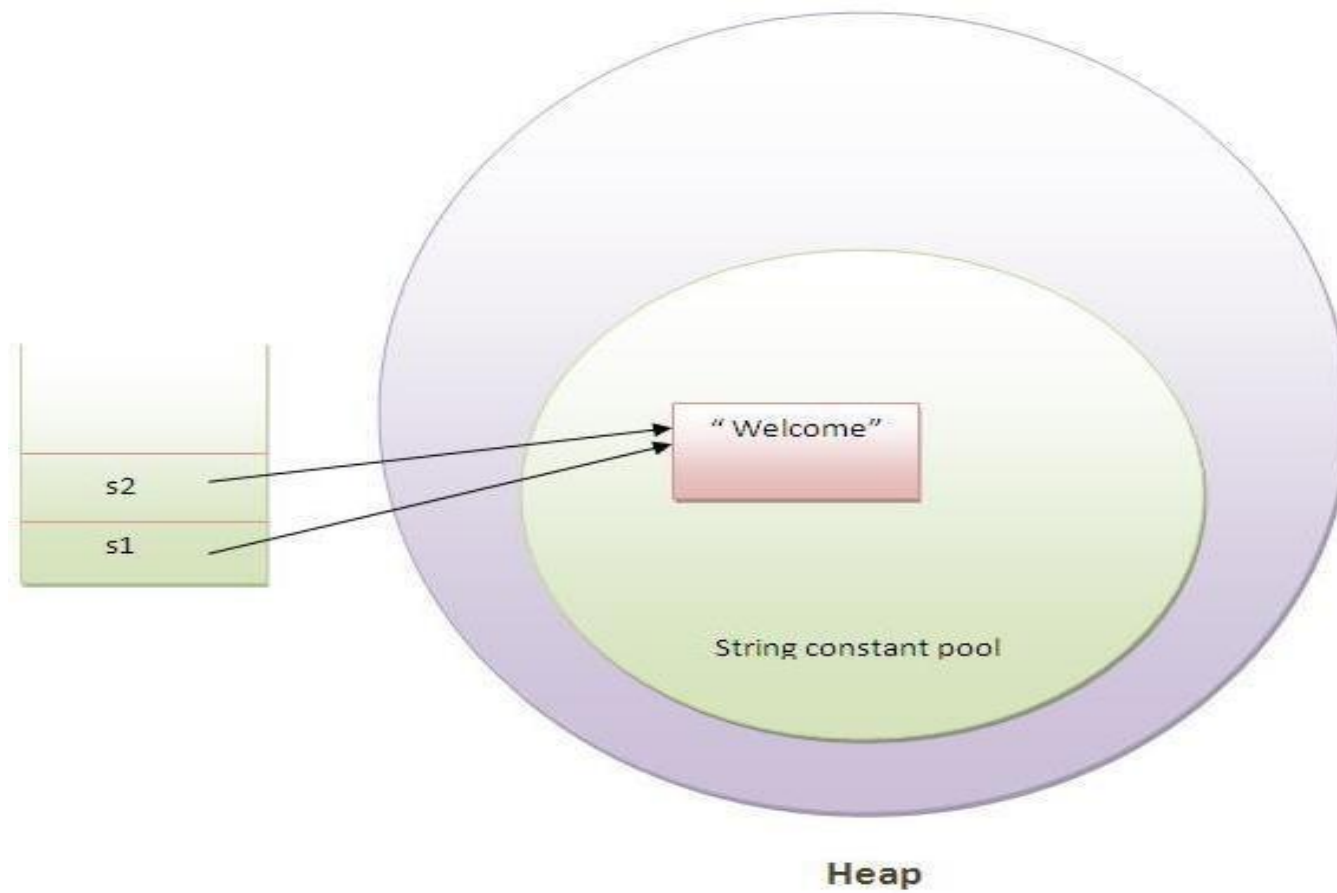
For example:

1. String s1="Welcome";
2. String s2="Welcome";//will not create new instance

Note: String objects are stored in a special memory area known as string constant pool.

Why java uses concept of string literal?

To make Java more memory efficient (because no new objects are created if it exists already in string constant pool).



In the above example only one object will be created. Firstly JVM will not find any string object with the value "Welcome" in string constant pool, so it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create new object but will return the reference to the same instance.

2) By new keyword

String s=**new** String("Welcome");//creates two objects and one reference variable

In such case

- JVM will create a new string object in normal(non pool) heap memory and the literal "Welcome" will be placed in the string constant pool.
- The variable s will refer to the object in heap(non pool).

Example

```
public class StringExample{  
public static void main(String args[]){  
String s1="java";//creating string by java string literal  
String s2=new String("example");//creating java string by new keyword  
  
char ch[]={'s','t','r','i','n','g','s'};  
String s3=new String(ch);//converting char array to string  
System.out.println(s1);  
System.out.println(s2);  
System.out.println(s3);  
} }
```

output:

```
java  
example  
strings
```

Immutable String in Java

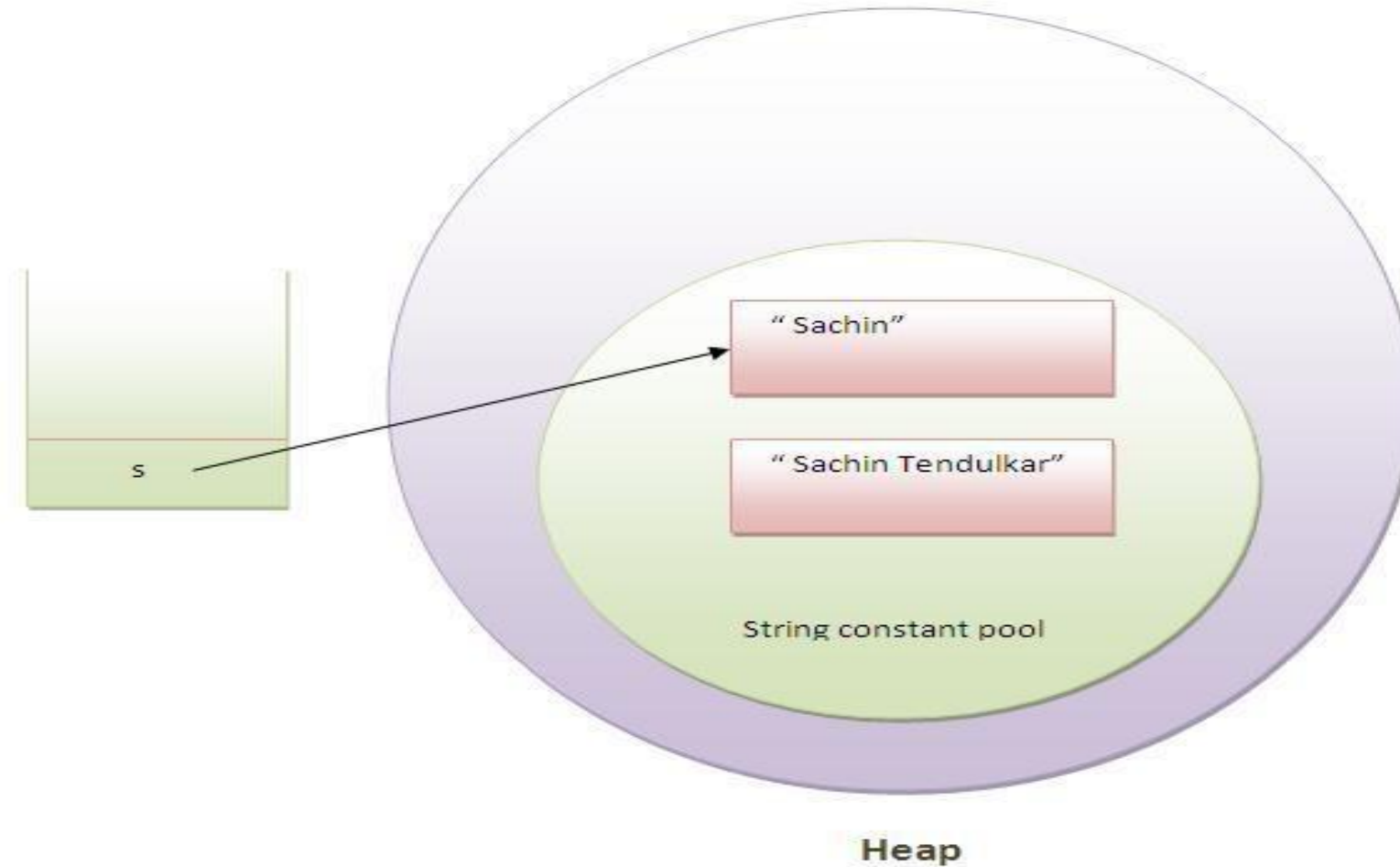
- In java, **string objects are immutable**. Immutable simply means unmodifiable or unchangeable.
- Once string object is created its data or state can't be changed but a new string object is created.

Example:

```
class Testimmutablestring {  
    public static void main(String args[]) {  
        String s="Sachin";  
        s.concat(" Tendulkar");//concat() method appends  
            the string at the end  
        System.out.println(s);//will print Sachin because  
            strings are immutable objects  
    }  
}
```

output:

Now it can be understood by the diagram given below. Here Sachin is not changed but a new object is created with sachintendulkar. That is why string is known as immutable.



As you can see in the above figure that two objects are created but s reference variable still refers to "Sachin" not to "Sachin Tendulkar".

But if we explicitly assign it to the reference variable, it will refer to "Sachin Tendulkar" object.

Example:

```
class Testimmutablestring1 {  
    public static void main(String args[]){  
        String s="Sachin";  
        s=s.concat(" Tendulkar");  
        System.out.println(s);  
    }  
}
```

Output:

Sachin Tendulkar

- In such case, s points to the "Sachin Tendulkar".
- Please notice that still sachin object is not modified.

Why string objects are immutable in java?

Because java uses the concept of string literal. Suppose there are 5 reference variables, all refers to one object "sachin". If one reference variable changes the value of the object, it will be affected to all the reference variables. That is why string objects are immutable in java.

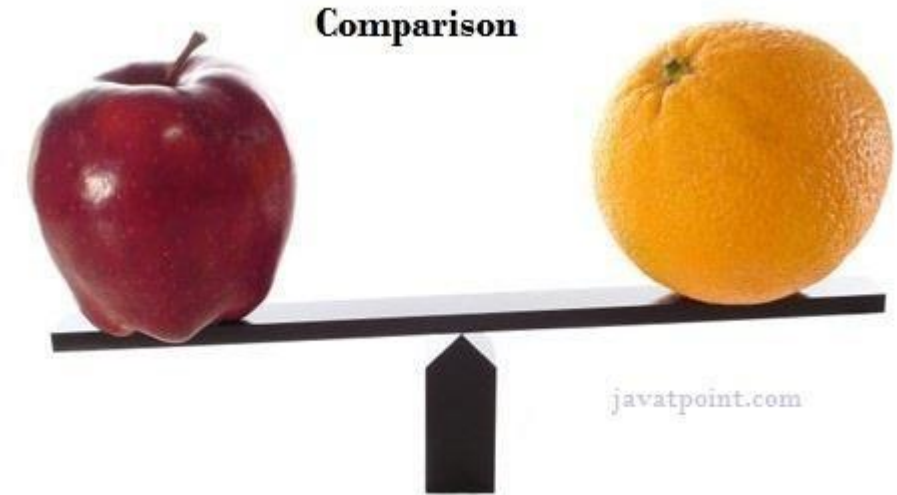
String comparison in Java

There are three ways to compare String objects:

1. By equals() method
2. By == operator
3. By compareTo() method

We can compare two given strings on the basis of content and reference.

Authentication (by equals() method),
sorting (by compareTo() method),
reference matching (by == operator)



1) By equals() method

- equals() method compares the original content of the string.
- It compares values of string for equality.

String class provides two methods:

`public boolean equals(Object another){}` compares this string to the specified object.

`public boolean equalsIgnoreCase(String another){}` compares this String to another String, ignoring case.

Example:

```
class Teststringcomparison1 {  
  public static void main(String args[]) {  
    String  
    s1="Sachin";  
    String  
    s2="Sachin";  
    String s3=new String("Sachin");  
    String s4="Saurav";  
    System.out.println(s1.equals(s2));//true  
    System.out.println(s1.equals(s3));//true  
    System.out.println(s1.equals(s4));//false  
  }  
}
```

output:

true

Example of equalsIgnoreCase(String) method

```
class Teststringcomparison2 {  
    public static void main(String args[]) {  
  
        String s1="Sachin";  
        String s2="SACHIN";  
  
        System.out.println(s1.equals(s2));//false  
        System.out.println(s1.equalsIgnoreCase(s3));//true  
    }  
}
```

output:

false

true

2) By == operator

The == operator compares references not values

Example:

```
class Teststringcomparison3 {  
    public static void main(String args[]) {  
        String s1="Sachin";  
        String s2="Sachin";  
        String s3=new String("Sachin");  
  
        System.out.println(s1==s2);//true (because both refer to same instance)  
        System.out.println(s1==s3);//false(because s3 refers to instance created i n nonpool)  
    }  
}
```

output:

true

false

3) By compareTo() method:

compareTo() method compares values and returns an int which tells if the values compare less than, equal, or greater than.

Suppose s1 and s2 are two string variables. If:

$s1 == s2$:0

$s1 > s2$:positive value

$s1 < s2$:negative value

Example of compareTo() method:

```
class Teststringcomparison4{  
public static void main(String args[]){  
String s1="Sachin";  
String s2="Sachin";  
String s3="Ratan";  
System.out.println(s1.compareTo(s2));  
  
System.out.println(s1.compareTo(s3));  
  
System.out.println(s3.compareTo(s1));  
}  
}
```

output:

0
1
-1

String Concatenation in Java

Concatinating strings form a new string i.e. the combination of multiple strings.

There are two ways to concat string objects:

- 1.By + (string concatenation) operator
- 2.By concat() method

1) By + (string concatenation) operator

String concatenation operator is used to add strings.

//Example of string concatenation operator

```
class TestStringConcatenation1 {  
    public static void main(String args[]) {  
  
        String s="Sachin"+" Tendulkar";  
        System.out.println(s);  
    }  
}
```

Output:

Sachin Tendulkar

The compiler transforms this to:

```
String s=(new StringBuilder()).append("Sachin").append(" Tendulkar").toString();
```

String concatenation is implemented through the `StringBuilder`(or `StringBuffer`) class and its `append` method.

String concatenation operator produces a new string by appending the second operand onto the end of the first operand.

The string concatenation operator can concat not only string but primitive values also.

Example:

```
class TestStringConcatenation2 {  
  
    public static void main(String args[]) {  
  
        String s=50+30+"Sachin"+40+40;  
  
        System.out.println(s); //80Sachin4040  
  
    }  
}
```

Output:

80Sachin4040

Note:

If either operand is a string, the resulting operation will be string concatenation.
If both operands are numbers, the operator will perform an addition.

2) By concat() method

concat() method concatenates the specified string to the end of current string.

Syntax: public String concat(String another){}

Example of concat(String) method

```
class TestStringConcatenation3 {  
    public static void main(String args[]) {  
        String s1="Sachin ";  
        String s2="Tendulkar";  
        String s3=s1.concat(s2);  
        System.out.println(s3);  
    }  
}
```

Output :

Sachin Tendulkar

2.String class methods and operations

- The java.lang.String class provides a lot of methods to work on string.
- By the help of these methods, we can perform operations on string such as trimming, concatenating, converting, comparing, replacing strings etc.
- Java String is a powerful concept because everything is treated as a string if you submit any form in window based, web based or mobile application.

String class methods

The java.lang.String class provides many useful methods to perform operations on sequence of char values.

No.	Method	Description
1	<u>char charAt(int index)</u>	returns char value for the particular index
2	<u>int length()</u>	returns string length
3	<u>static String format(String format, Object... args)</u>	returns formatted string
4	<u>static String format(Locale l, String format, Object... args)</u>	returns formatted string with given locale
5	<u>String substring(int beginIndex)</u>	returns substring for given begin index
6	<u>String substring(int beginIndex, int endIndex)</u>	returns substring for given begin index and end index
7	<u>boolean contains(CharSequence s)</u>	returns true or false after matching the sequence of char value

8	<u>static String join(CharSequence delimiter, CharSequence... elements)</u>	returns a joined string
9	<u>static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)</u>	returns a joined string
10	<u>boolean equals(Object another)</u>	checks the equality of string with object
11	<u>boolean isEmpty()</u>	checks if string is empty
12	<u>String concat(String str)</u>	concatinates specified string
13	<u>String replace(char old, char new)</u>	replaces all occurrences of specified char value
14	<u>String replace(CharSequence old, CharSequence new)</u>	replaces all occurrences of specified CharSequence
15	<u>String trim()</u>	returns trimmed string omitting leading and trailing spaces
16	<u>String split(String regex)</u>	returns splitted string matching regex
17	<u>String split(String regex, int limit)</u>	returns splitted string matching regex and limit

18	<u>String intern()</u>	returns interned string
19	<u>int indexOf(int ch)</u>	returns specified char value index
20	<u>int indexOf(int ch, int fromIndex)</u>	returns specified char value index starting with given index
21	<u>int indexOf(String substring)</u>	returns specified substring index
22	<u>int indexOf(String substring, int fromIndex)</u>	returns specified substring index starting with given index
23	<u>String toLowerCase()</u>	returns string in lowercase.
24	<u>String toLowerCase(Locale l)</u>	returns string in lowercase using specified locale.
25	<u>String toUpperCase()</u>	returns string in uppercase.
26	<u>String toUpperCase(Locale l)</u>	returns string in uppercase using specified locale.

charAt() Method

The **java string charAt()** method returns a char value at the given index number. The index number starts from 0.

Signature

```
public char charAt(int index)
```

Parameter

index : index number, starts with 0

Returns

char value

Specified by

CharSequence interface

Throws

IndexOutOfBoundsException

ion : if index is negative value or greater than this string length.

String charAt() method example

```
public class CharAtExample{  
public static void main(String args[]){  
String name="javat";  
char ch=name.charAt(4);//returns the char value at the 4th index  
  
System.out.println(ch);  
}  
}
```

Output: t

String concat() Method

The java string concat() method combines specified string at the end of this string.

Signature:

```
public String concat(String anotherString)
```

Parameter

anotherString : another string i.e. to be combined at the end of this string.

Returns

combined string

String concat() method example

```
public class ConcatExample{  
public static void main(String args[]){  
String s1="java string";  
s1.concat("is immutable");  
System.out.println(s1);  
s1=s1.concat(" is immutable so assign it explicitly");  
System.out.println(s1);  
}  
}
```

Output: java string

java string is immutable so assign it explicitly

String contains() Method

The java string contains() method searches the sequence of characters in this string. It returns true if sequence of char values are found in this string otherwise returns false.

Signature:

```
public boolean contains(CharSequence sequence)
```

Parameter

sequence : specifies the sequence of characters to be searched.

Returns

true if sequence of char value exists, otherwise false.

Throws

NullPointerException : if sequence is null.

String contains() method example

```
class ContainsExample{  
public static void main(String args[]){  
String name="what do you know about me";  
System.out.println(name.contains("do you know"));  
System.out.println(name.contains("about"));  
System.out.println(name.contains("hello"));  
}  
}
```

Output:

true

true

false

String equals() Method

The java string equals() method compares the two given strings based on the content of the string. If any character is not matched, it returns false. If all characters are matched, it returns true.

The String equals() method overrides the equals() method of Object class.

Signature

```
public boolean equals(Object anotherObject)
```

Parameter

anotherObject : another object i.e. compared with this string.

Returns

true if characters of both strings are equal otherwise false.

Overrides

equals() method of java Object class.

String equals() method example

```
public class EqualsExample{  
public static void main(String args[]){  
String s1="java";  
String s2="java";  
String s3="JAVACLASS";  
String s4="python";  
System.out.println(s1.equals(s2));//true because content and case is same  
System.out.println(s1.equals(s3));//false because case is not same  
System.out.println(s1.equals(s4));//false because content is not same  
}  
}
```

Output:

```
true  
false  
false
```

String format() method

The java string format() method returns the formatted string by given locale, format and arguments. If you don't specify the locale in String.format() method, it uses default locale by calling Locale.getDefault() method.

The format() method of java language is like sprintf() function in c language and printf() method of java language.

Signature: Two types of signatures

public static String format(String format, Object... args) and,

public static String format(Locale locale, String format, Object... args)

Parameters

locale : specifies the locale to be applied on the format() method.

format : format of the string.

args : arguments for the format string. It may be zero or more.

Returns

formatted

string

Throws

NullPointerException : if format is null **and** IllegalFormatException : if format is illegal or

String format() method example

```
public class FormatExample{  
public static void main(String args[]){  
String name="nitin";  
String sf1=String.format("name is %s",name);  
String sf2=String.format("value is %f",32.33434);  
String sf3=String.format("value is %32.12f",32.33434);//returns 12 char fractional part filling  
with 0  
System.out.println(sf1);  
System.out.println(sf2);  
System.out.println(sf3);  
}} output:
```

name is nitin

value is 32.334340

value is 32.33434000000000

String `getBytes()` Method

The **java string `getBytes()`** method returns the byte array of the string. In other words, it returns sequence of bytes.

Signature

There are 3 variant of `getBytes()` method. The signature or syntax of string `getBytes()` method is given below:

1. **public byte[]** `getBytes()`
2. **public byte[]** `getBytes(Charset charset)`
3. **public byte[]** `getBytes(String charsetName)`**throws** `UnsupportedEncodingException`

Returns

- sequence of bytes.

Java String getBytes() method example

```
public class StringGetBytesExample{  
public static void main(String args[]){  
String s1="ABCDEFGF";  
byte[] barr=s1.getBytes();  
for(int i=0;i<barr.length;i++){  
    System.out.println(barr[i]);  
}}}
```

output:

65

66

67

68

69

70

71

String indexOf() method

The java string indexOf() method will give index of given character value or substring. If it is not found, it returns -1. The index counter starts from zero.

Parameters

ch: char value i.e. a single character e.g. 'a'

fromIndex: index position from where index of the char value or substring is returned

substring: substring to be searched in this string

Returns

index of the string or char

Signature

There are 4 types of indexOf method in java. The signature of indexOf methods are given below:

No.	Method	Description
1	int indexOf(int ch)	returns index position for the given char value
2	int indexOf(int ch, int fromIndex)	returns index position for the given char value and from index
3	int indexOf(String substring)	returns index position for the given substring
4	int indexOf(String substring, int fromIndex)	returns index position for the given substring and from index

String indexOf() method example

```
public class IndexOfExample{  
public static void main(String args[]){  
String s1="this is index of example";  
//passing substring  
int index1=s1.indexOf("is");//returns the index of is substring  
int index2=s1.indexOf("index");//returns the index of index substring  
System.out.println(index1+" "+index2);//2 8  
//passing substring with from index  
int index3=s1.indexOf("is",4);//returns the index of is substring after 4th index  
System.out.println(index3);//5 i.e. the index of another is  
//passing char value  
int index4=s1.indexOf('s');//returns the index of s char value  
System.out.println(index4);//3  
}}
```

output:

2 8

5

3

String intern() Method

The java string intern() method returns the interned string. It returns the canonical representation of string.

It can be used to return string from pool memory, if it is created by new keyword.

Signature

```
public String intern()
```

Returns

interned string

String intern() method example

```
public class InternExample{  
public static void main(String args[]){  
String s1=new String("hello");  
String s2="hello";  
String s3=s1.intern();//returns string from pool, now it will be same as s2  
System.out.println(s1==s2);//false because reference is different  
  
System.out.println(s2==s3);//true because reference is same  
}  
}
```

Output:

false

true

String isEmpty() Method

The java string isEmpty() method checks if this string is empty. It returns true, if length of string is 0 otherwise false.

Signature

```
public boolean isEmpty()
```

Returns

true if length is 0 otherwise false.

String isEmpty() method example

```
public class IsEmptyExample{  
public static void main(String args[]){  
    String s1="";  
    String s2="java";  
  
    System.out.println(s1.isEmpty());  
    System.out.println(s2.isEmpty());  
}  
}
```

Output:

true

false

String join() Method

The java string join() method returns a string joined with given delimiter.

In string join method, delimiter is copied for each elements.

In case of null element, "null" is added.

The join() method is included in java string since JDK 1.8.

There are two types of join() methods in java string.

Signature

public static String join(CharSequence delimiter, CharSequence... elements) and

public static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)

Parameters

delimiter : char value to be added with each element

elements : char value to be attached with delimiter

Returns

joined string with delimiter

Throws

NullPointerException if element or delimiter is null.

String join() method example

```
public class StringJoinExample{  
public static void main(String args[]){  
String joinString1=String.join("-","welcome","to","java");  
System.out.println(joinString1);  
}  
}
```

Output:

welcome-to-java

String length() Method

The java string length() method length of the string. It returns count of total number of characters.

The length of java string is same as the unicode code units of the string.

Signature

```
public int length()
```

Specified by

CharSequence interface

Returns

length of characters

String length() method example

```
public class LengthExample{  
    public static void main(String args[]){  
        String s1="java";  
        String s2="python";  
  
        System.out.println("string length is: "+s1.length());//4 is the length of java string  
        System.out.println("string length is: "+s2.length());//6 is the length of python string  
    }  
}
```

Output:

```
string length is: 4  
string length is: 6
```

String replace() Method

The java string replace() method returns a string replacing all the old char or CharSequence to new char or CharSequence.

replace() method will allow you to replace a sequence of char values.

Signature: *Two methods*

public String replace(char oldChar, char newChar) and

public String replace(CharSequence target, CharSequence replacement)

The second replace method is added since JDK 1.5.

Parameters

oldChar : old character

newChar : new character

target : target sequence of characters

replacement : replacement sequence of characters

Returns

replaced string

String replace(char old, char new) method example

```
public class ReplaceExample1 {  
public static void main(String args[]){  
String s1="java";  
  
String replaceString=s1.replace('a','e');//replaces all occurrences of 'a' to 'e'  
  
System.out.println(replaceString);  
}  
}
```

Output :

jeve

String replace(CharSequence target, CharSequence replacement) method example

```
public class ReplaceExample2 {  
public static void main(String args[]) {  
String s1="my name is khan my name is java";  
String replaceString=s1.replace("is","was");//replaces all occurrences of "is" to "was"  
System.out.println(replaceString);  
}  
}
```

Output:

my name was khan my name was java

String split() Method

The java string split() method splits this string against given regular expression and returns a char array.

Signature

There are two signature for split() method in java string.

```
public String split(String regex)
```

```
public String split(String regex, int limit)
```

Parameter

regex : regular expression to be applied on string.

limit : limit for the number of strings in array. If it is zero, it will return all the strings matching regex.

Returns

array of strings

Throws

PatternSyntaxException if pattern for regular expression is invalid

String split() method example

The given example returns total number of words in a string excluding space only. It also includes special characters.

```
public class SplitExample{  
public static void main(String args[]){  
    String s1="java string split method";  
    String[] words=s1.split("\\s");//splits the string based on string  
    //using java foreach loop to print elements of string array  
    for(String w:words){  
        System.out.println(w);  
    }  
}
```

output:

```
java  
string  
split  
method
```

String split() method with regex and length example

```
public class SplitExample2 {  
    public static void main(String args[]) {  
        String s1="welcome to split world";  
        System.out.println("returning words:");  
        for(String w:s1.split("\\s",0)){  
            System.out.println(w);  
        }  
        System.out.println("returning words:");  
        for(String w:s1.split("\\s",1)){  
            System.out.println(w);  
        }  
        System.out.println("returning words:");  
        for(String :s1.split("\\s",2)){  
            System.out.println(w);  
        }  
    }  
}
```

```
}  
}  
}
```

output:

returning words:

welcome

to

split

world

returning words:

welcome to split

world

returning words:

welcome to split

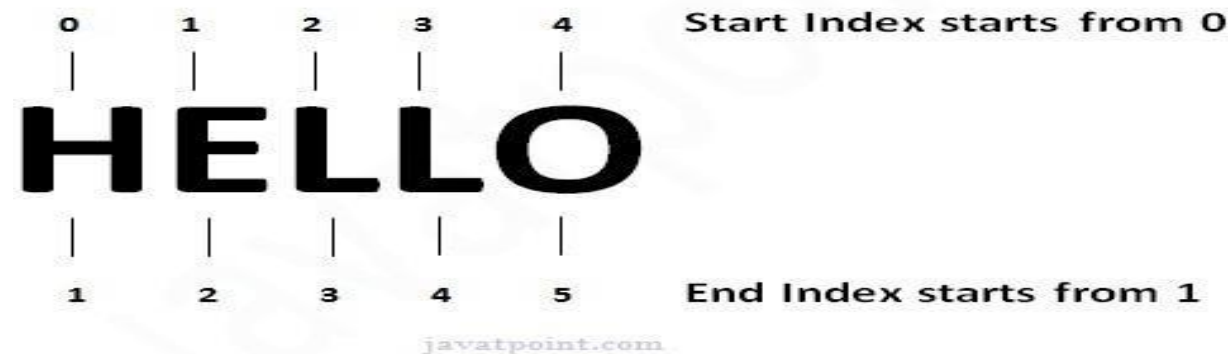
world

Substring in Java

A part of string is called **substring**. In other words, substring is a subset of another string. In case of substring `startIndex` starts from 0 and `endIndex` starts from 1 or `startIndex` is inclusive and `endIndex` is exclusive.

You can get substring from the given String object by one of the two methods:

- 1. `public String substring(int startIndex)`:** This method returns new String object containing the substring of the given string from specified `startIndex` (inclusive).
- 2. `public String substring(int startIndex, int endIndex)`:** This method returns new String object containing the substring of the given string from specified `startIndex` to `endIndex`.



String substring() Method

The java string substring() method returns a part of the string.

We pass begin index and end index number position in the java substring method where start index is inclusive and end index is exclusive. In other words, start index starts from 0 whereas end index starts from 1.

There are two types of substring methods in java string.

Signature

```
public String substring(int startIndex)
```

```
public String substring(int startIndex, int endIndex)
```

If you don't specify endIndex, java substring() method will return all the characters from startIndex.

Parameters

startIndex : starting index is inclusive

endIndex : ending index is exclusive

Returns

specified

string

Throws

StringIndexOutOfBoundsException if start index is negative value or end index is lower than

In case of string:

startIndex:starts from index 0(inclusive).

endIndex:starts from index 1(exclusive).

//Example of substring() method

```
public class TestSubstring{  
    public static void main(String args[]){  
        String s="Sachin Tendulkar";  
        System.out.println(s.substring(6));//Tendulkar  
        System.out.println(s.substring(0,6));//Sachin  
    }  
}
```

Output:

Tendulkar

Sachin

String substring() method example

```
public class SubstringExample{  
public static void main(String args[]){  
String s1="javahello";  
  
System.out.println(s1.substring(2,4));//returns va  
  
System.out.println(s1.substring(2));//returns vahello  
}  
}
```

Output:

va

vahello

String toLowerCase() method

The java string toLowerCase() method returns the string in lowercase letter. In other words, it converts all characters of the string into lower case letter.

The toLowerCase() method works same as toLowerCase(Locale.getDefault()) method. It internally uses the default locale.

Signature

There are two variant of toLowerCase() method.

```
public String toLowerCase()
```

```
public String toLowerCase(Locale locale)
```

The second method variant of toLowerCase(), converts all the characters into lowercase using the rules of given Locale.

Returns

string in lowercase letter.

String toLowerCase() method example

```
public class StringLowerExample{  
public static void main(String args[]){  
    String s1="JAVA HELLO stRIng";  
  
    String lower=s1.toLowerCase();  
  
    System.out.println(lower);  
}  
}
```

Output:

java hello string

String toUpperCase() method

The java string toUpperCase() method returns the string in uppercase letter. In other words, it converts all characters of the string into upper case letter.

The toUpperCase() method works same as toUpperCase(Locale.getDefault()) method. It internally uses the default locale.

Signature

There are two variant of toUpperCase() method.

```
public String toUpperCase()
```

```
public String toUpperCase(Locale locale)
```

The second method variant of toUpperCase(), converts all the characters into uppercase using the rules of given Locale.

Returns

string in uppercase letter.

String toUpperCase() method example

```
public class StringUpperExample{  
public static void main(String args[]){  
String s1="hello string";  
String upper=s1.toUpperCase();  
System.out.println(upper);  
}  
}
```

Output:

HELLO STRING

String trim() method

The java string trim() method eliminates leading and trailing spaces.

The unicode value of space character is '\u0020'.

The trim() method in java string checks this unicode value before and after the string, if it exists then removes the spaces and returns the omitted string.

The string trim() method doesn't omit middle spaces.

Signature

```
public String trim()
```

Returns

string with omitted leading and trailing spaces

String trim() method example

```
public class StringTrimExample{  
    public static void main(String args[]){  
        String s1="        hello string        ";  
  
        System.out.println(s1+"java");//without trim()  
        System.out.println(s1.trim()+"java");//with trim()  
    }  
}
```

output:

```
hello string        java  
hello stringjava
```

String valueOf() method

The string valueOf() method converts given type such as int, long, float, double, boolean, char and char array into string.

```
public class StringTrimExample{  
    public static void main(String args[]){  
        int a=10;  
        String s=String.valueOf(a);  
        System.out.println(s+10);  
    }  
}
```

Output:

1010

Understanding toString() method

If you want to represent any object as a string, **toString() method** comes into existence.

The toString() method returns the string representation of the object.

If you print any object, java compiler internally invokes the toString() method on the object.

So overriding the toString() method, returns the desired output, it can be the state of an object etc. depends on your implementation.

Advantage of the toString() method

By overriding the toString() method of the Object class, we can return values of the object, so we don't need to write much code.

Understanding problem without toString() method

```
class Student{
    int rollno; String name; String city;
    Student(int rollno, String name, String city){
        this.rollno=rollno;
        this.name=name;
        this.city=city;
    }
    public static void main(String args[]){
        Student s1=new Student(101,"Raj","lucknow");
        Student s2=new Student(102,"Vijay","ghaziabad");
        System.out.println(s1);//compiler writes here s1.toString()
        System.out.println(s2);//compiler writes here s2.toString()
    }}

```

Output:

Student@1fee6fc

Student@1eed786

As you can see in the above example, printing s1 and s2 prints the hashcode values of the objects but I want to print the values of these objects. Since java compiler internally calls toString() method, overriding this method will return the specified values.

Let's understand it with the example given below:

Example of toString() method

```
class Student{  
    int rollno; String name; String city;  
    Student(int rollno, String name, String city){  
        this.city=city;  
        this.rollno=rollno;  
        this.name=name;  
    }  
    public String toString(){//overriding the toString() method  
        return rollno+" "+name+" "+city;  
    }  
    public static void main(String args[]){  
        Student s1=new Student(101,"Raj","lucknow");  
        Student s2=new Student(102,"Vijay","ghaziabad");  
        System.out.println(s1);//compiler writes here s1.toString()  
        System.out.println(s2);//compiler writes here s2.toString()  
    }  
}
```

Output:

101 Raj lucknow

102 Vijay ghaziabad

3.StringBuffer class

- The StringBuffer class is used to create mutable (modifiable) string.
- The StringBuffer class is same as String except it is mutable i.e. it can be changed.

Note: StringBuffer class is thread-safe i.e. multiple threads cannot access it simultaneously. So it is safe and will result in an order.

Commonly used Constructors of StringBuffer class:

StringBuffer(): creates an empty string buffer with the initial capacity of 16.

StringBuffer(String str): creates a string buffer with the specified string.

StringBuffer(int capacity): creates an empty string buffer with the specified capacity as length.

Commonly used methods of StringBuffer class:

➤ **public synchronized StringBuffer append(String s):**

It is used to append the specified string with this string. The append() method is overloaded like append(char), append(boolean), append(int), append(float), append(double) etc.

➤ **public synchronized StringBuffer insert(int offset, String s):**

It is used to insert the specified string with this string at the specified position. The insert() method is overloaded like insert(int, char), insert(int, boolean), insert(int, int), insert(int, float), insert(int, double) etc.

➤ **public synchronized StringBuffer replace(int startIndex, int endIndex, String str):**

It is used to replace the string from specified startIndex and endIndex.

➤ **public synchronized StringBuffer delete(int startIndex, int endIndex):**

It is used to delete the string from specified startIndex and endIndex.

- **public synchronized StringBuffer reverse():** is used to reverse the string.
- **public int capacity():** is used to return the current capacity.
- **public void ensureCapacity(int minimumCapacity):** is used to ensure the capacity at least equal to the given minimum.
- **public char charAt(int index):** is used to return the character at the specified position.
- **public int length():** is used to return the length of the string i.e. total number of characters.
- **public String substring(int beginIndex):** is used to return the substring from the specified beginIndex.
- **public String substring(int beginIndex, int endIndex):** is used to return the substring from the specified beginIndex and endIndex.

What is mutable string?

A string that can be modified or changed is known as mutable string. StringBuffer and StringBuilder classes are used for creating mutable string.

Example of StringBuffer class by append() method

The append() method concatenates the given argument with this string.

```
class A{  
    public static void main(String args[]){  
  
        StringBuffer sb=new StringBuffer("Hello ");  
        sb.append("Java");//now original string is changed  
        System.out.println(sb);//Hello Java  
    }  
}
```

Output:

Hello Java

Example of insert() method of StringBuffer class

The insert() method inserts the given string with this string at the given position.

```
class A{  
public static void main(String args[]){  
    StringBuffer sb=new StringBuffer("Hello ");  
    sb.insert(1, "Java");//now original string is changed  
    System.out.println(sb);//prints HJavaello  
}  
}
```

output:

HJavaello

Example of replace() method of StringBuffer class

The replace() method replaces the given string from the specified beginIndex and endIndex.

```
class A {  
    public static void main(String args[]) {  
        StringBuffer sb=new StringBuffer("Hello");  
        sb.replace(1,3,"Java");  
        System.out.println(sb); //prints HJavallo  
    }  
}
```

Output:

HJavallo

Example of delete() method of StringBuffer class

The delete() method of StringBuffer class deletes the string from the specified beginIndex to endIndex.

```
class A {  
    public static void main(String args[]) {  
        StringBuffer sb=new StringBuffer("Hello");  
        sb.delete(1,3); System.out.println(sb); //prints Hlo  
    }  
}
```

Output:

Hlo

Example of reverse() method of StringBuffer class

The reverse() method of StringBuffer class reverses the current string.

```
class A {  
  public static void main(String args[]) {  
    StringBuffer sb=new StringBuffer("Hello");  
    sb.reverse();  
    System.out.println(sb);//prints olleH  
  }  
}
```

output:

olleH

Example of capacity() method of StringBuffer class

The capacity() method of StringBuffer class returns the current capacity of the buffer.

The default capacity of the buffer is 16.

If the number of character increases from its current capacity, it increases the capacity by $(\text{oldcapacity} * 2) + 2$.

For example if your current capacity is 16, it will be $(16 * 2) + 2 = 34$.

```
class A{
```

```
public static void main(String args[]){
```

```
    StringBuffer sb=new StringBuffer();
```

```
    System.out.println(sb.capacity());//default 16
```

```
    sb.append("Hello");
```

```
    System.out.println(sb.capacity());//now 16
```

```
    sb.append("java is my favourite language");
```

```
    System.out.println(sb.capacity());//now  $(16 * 2) + 2 = 34$  i.e  $(\text{oldcapacity} * 2) + 2$ 
```

```
} }
```

Example of ensureCapacity() method of StringBuffer class

The ensureCapacity() method of StringBuffer class ensures that the given capacity is the minimum to the current capacity. If it is greater than the current capacity, it increases the capacity by $(\text{oldcapacity} * 2) + 2$. For example if your current capacity is 16, it will be $(16 * 2) + 2 = 34$.

```
class A{  
    public static void main(String args[]){  
        StringBuffer sb=new StringBuffer();  
        System.out.println(sb.capacity());//default 16  
        sb.append("Hello");  
        System.out.println(sb.capacity());//now 16  
        sb.append("java is my favourite language");  
        System.out.println(sb.capacity());//now  $(16 * 2) + 2 = 34$  i.e  $(\text{oldcapacity} * 2) + 2$   
        sb.ensureCapacity(10);//now no change  
        System.out.println(sb.capacity());//now 34  
        sb.ensureCapacity(50);//now  $(34 * 2) + 2$   
        System.out.println(sb.capacity());//now 70  }}
```

How to create Immutable class?

There are many immutable classes like String, Boolean, Byte, Short, Integer, Long, Float, Double etc. In short, all the wrapper classes and String class is immutable.

We can also create immutable class by creating final class that have final data members as the example given below:

Example to create Immutable class:

In the example, we have created a final class named Employee. It have one final data member, a parameterized constructor and getter method.

```
public final class Employee{  
    final String pancardNumber;  
    public Employee(String pancardNumber){  
        this.pancardNumber=pancardNumber;  
    }  
    public String getPancardNumber(){  
        return pancardNumber;  
    }  
}
```

- The above class is immutable because:
- The instance variable of the class is final i.e. we cannot change the value of it after creating an object.
- The class is final so we cannot create the subclass.
- There is no setter methods i.e. we have no option to change the value of the instance variable.
- These points makes this class as immutable.

StringBuilder class:

The StringBuilder class is used to create mutable (modifiable) string. The StringBuilder class is same as StringBuffer class except that it is non-synchronized. It is available since JDK1.5.

Commonly used Constructors of StringBuilder class:

StringBuilder(): creates an empty string Builder with the initial capacity of 16.

StringBuilder(String str): creates a string Builder with the specified string.

StringBuilder(int length): creates an empty string Builder with the specified capacity as length.

• Commonly used methods of StringBuilder class:

1. public StringBuilder append(String s): is used to append the specified string with this string. The append() method is overloaded like append(char), append(boolean), append(int), append(float), append(double) etc.

2. public StringBuilder insert(int offset, String s): is used to insert the specified string with this string at the specified position. The insert() method is overloaded like insert(int, char), insert(int, boolean), insert(int, int), insert(int, float), insert(int, double) etc.

3. public StringBuilder replace(int startIndex, int endIndex, String str): is used to replace the string from specified startIndex and endIndex.

4. public StringBuilder delete(int startIndex, int endIndex): is used to delete the string from specified startIndex and endIndex.

5. public StringBuilder reverse(): is used to reverse the string.

6. public int capacity(): is used to return the current capacity.

7. public void ensureCapacity(int minimumCapacity): is used to ensure the capacity at least equal to the given minimum.

8. public char charAt(int index): is used to return the character at the specified position.

9. public int length(): is used to return the length of the string i.e. total number of characters.

10. public String substring(int beginIndex): is used to return the substring from the specified beginIndex.

11. public String substring(int beginIndex, int endIndex): is used to return the substring from the specified beginIndex and endIndex.

simple program of StringBuilder class by append() method

The append() method concatenates the given argument with this string.

```
class A{  
public static void main(String args[]){  
  
    StringBuilder sb=new StringBuilder("Hello ");  
    sb.append("Java");//now original string is changed  
  
    System.out.println(sb);//  
}  
}
```

Example of insert() method of StringBuilder class

The insert() method inserts the given string with this string at the given position.

```
class A{
```

```
public static void main(String args[]){
```

```
StringBuilder sb=new StringBuilder("Hello ");
```

```
sb.insert(1, "Java");//now original string is changed
```

```
System.out.println(sb);//
```

```
}
```

```
}
```

Example of replace() method of StringBuilder class

The replace() method replaces the given string from the specified beginIndex and endIndex.

```
class A{  
public static void main(String args[]){  
  
    StringBuilder sb=new StringBuilder("Hello");  
    sb.replace(1,3,"Java");  
  
    System.out.println(sb);//}  
}
```

Example of delete() method of StringBuilder class

The delete() method of StringBuilder class deletes the string from the specified beginIndex to endIndex.

```
class A{  
public static void main(String args[]){  
  
    StringBuilder sb=new StringBuilder("Hello");  
    sb.delete(1,3);  
  
    System.out.println(sb);  
}  
}
```

Example of reverse() method of StringBuilder class

The reverse() method of StringBuilder class reverses the current string.

```
class A{  
public static void main(String args[]){  
  
    StringBuilder sb=new StringBuilder("Hello");  
    sb.reverse();  
  
    System.out.println(sb);//}  
}
```

Example of capacity() method of StringBuilder class

The capacity() method of StringBuilder class returns the current capacity of the Builder.

The default capacity of the Builder is 16. If the number of character increases from its current capacity, it increases the capacity by $(oldcapacity * 2) + 2$.

For example if your current capacity is 16, it will be $(16 * 2) + 2 = 34$.

```
class A{  
public static void main(String args[]){ StringBuilder sb=new  
StringBuilder(); System.out.println(sb.capacity());//default  
sb.append("Hello"); System.out.println(sb.capacity());//now  
sb.append("java is my favourite language");  
System.out.println(sb.capacity());//  
}  
}
```

Example of ensureCapacity() method of StringBuilder class

The ensureCapacity() method of StringBuilder class ensures that the given capacity is the minimum to the current capacity. If it is greater than the current capacity, it increases the capacity by $(\text{oldcapacity} * 2) + 2$. For example if your current capacity is 16, it will be $(16 * 2) + 2 = 34$.

```
class A{  
  
    public static void main(String args[]){  
  
        StringBuilder sb=new StringBuilder();  
        System.out.println(sb.capacity());//default  
        sb.append("Hello");  
        System.out.println(sb.capacity());//now  
        sb.append("java is my favourite language");  
        System.out.println(sb.capacity());//now  
        sb.ensureCapacity(10);//now  
        System.out.println(sb.capacity());//now  
        sb.ensureCapacity(50);//now  
        System.out.println(sb.capacity());//now  
    } }  
}
```

How to create Immutable class?

There are many immutable classes like String, Boolean, Byte, Short, Integer, Long, Float, Double etc. In short, all the wrapper classes and String class is immutable. We can also create immutable class by creating final class that have final data members as the example given below:

Example to create Immutable class:

In the next example, we have created a final class named Employee. It have one final data member, a parameterized constructor and getter method.

```
public final class Employee{  
final String pancardNumber;  
    public Employee(String pancardNumber){  
this.pancardNumber=pancardNumber;  
}  
    public String getPancardNumber(){  
return pancardNumber;  
} }
```

- The above class is immutable because: The instance variable of the class is final i.e. we cannot change the value of it after creating an object.
- The class is final so we cannot create the subclass.
- There is no setter methods i.e. we have no option to change the value of the instance variable.
- These points makes this class as immutable.

Understanding toString() method

If you want to represent any object as a string, **toString() method** comes into existence.

The toString() method returns the string representation of the object.

If you print any object, java compiler internally invokes the toString() method on the object. So overriding the toString() method, returns the desired output, it can be the state of an object etc. depends on your implementation.

Advantage of the toString() method

By overriding the toString() method of the Object class, we can return values of the object, so we don't need to write much code.

- **Understanding problem without toString() method**

Let's see the simple code that prints reference.

Output:Student@1fee6fc

Student@1eed786

As you can see in the above example, printing s1 and s2 prints the hashCode values of the objects but I want to print the values of these objects. Since java compiler internally calls toString() method, overriding this method will return the specified values. Let's understand it with the example given below:

- Example of toString() method

Now let's see the real example of toString() method.

```
class Student{ int rollno; String name;  
String city;  
    Student(int rollno, String name, String city){  
this.rollno=rollno;  
this.name=name;  
this.city=city;  
}  
    public String toString(){//overriding the toString() method  
return rollno+" "+name+" "+city;  
}  
public static void main(String args[]){  
    Student s1=new Student(101,"Raj","lucknow");  
    Student s2=new Student(102,"Vijay","ghaziabad");  
        System.out.println(s1);//compiler writes here s1.toString()  
        System.out.println(s2);//compiler writes here s2.toString()  
    }  
}
```

Output:101 Raj lucknow
102 Vijay ghaziabad